EMF Envelope FFT Analyzer

User Manual

Envelope (modulation) spectra of broadband EMF probe logs
(FieldMan / Narda-TSX, NBM-550, EHP-50 TS) and SignalShark spectrograms

Program version 0.0.2 · vszh5_writer 0.0.11 · 2026-10-15

William W. Wallace
NRAO/GBO
wwallace@nrao.edu

Copyright © 2026 William W. Wallace (Adaptive Electrodynamics LLC dba AdaptEd).
This manual and the programs it describes are free software under the
GNU General Public License v3.0 or later (GPL-3.0-or-later).

Internal draft: tested on synthetic files only (Section 11).

Contents

1 Introduction **4**
1.1 What the analysis can and cannot tell you . . . 4
1.2 Status of this version . . . 4
1.3 Conventions . . . 4

2 Installation **4**
2.1 Mamba environment . . . 4
2.2 Package files . . . 5
2.3 Veusz and LuaLaTeX (optional) . . . 5
2.4 Spyder . . . 5

3 Quick start **6**
3.1 Typical workflow . . . 6

4 Instruments and input files **6**
4.1 Probe catalogue . . . 6
4.2 Readers . . . 7
4.3 How text exports are read . . . 7
4.4 EHP-50 TS spectra . . . 8
4.5 SignalShark files . . . 8
4.6 Overriding the detection . . . 8

5 Theory and methods **9**
5.1 Square-law detection adds powers . . . 9
5.2 Why the transform is taken of power . . . 9
5.3 Unit conversions . . . 10
5.4 Sampling, probe averaging and aliasing . . . 10
5.5 Putting the log on a uniform grid . . . 10
5.6 Amplitude spectrum . . . 11
5.7 Welch power spectral density . . . 11
5.8 Chirp-Z zoom . . . 11
5.9 Lomb–Scargle periodogram . . . 12
5.10 Envelope spectrogram . . . 12
5.11 Probe correction . . . 12
5.12 Peaks and their exact frequency . . . 13
5.13 Harmonic families and rotation rate . . . 13
5.14 Spectrograms: envelope spectrum per frequency bin . . . 13

6 Command-line reference **13**

7 Output files **14**
7.1 The Veusz document . . . 15
7.2 Rendering to PDF and PNG . . . 15
7.3 Summary file . . . 16
7.4 Examples . . . 16

8 Live mode **20**

9 Memory, parallel processing and GPU **20**

10 Working in Spyder **20**

11 Verification **22**

12 Limitations and known issues **23**

13 Troubleshooting **23**

A Python modules used **25**
A.1 NumPy . . . 25
A.2 SciPy . . . 26
A.3 pandas and openpyxl . . . 26
A.4 h5py . . . 27
A.5 vszh5_writer (the author's module, 0.0.11) . . . 27
A.6 Qt.py, PySide6 and pyqtgraph . . . 28
A.7 CuPy (optional) . . . 28
A.8 pypdf and Veusz (rendering only) . . . 29
A.9 Python standard library . . . 29

B Revision history **29**

References **31**

List of Tables

1 Files of the package. . . . 5
2 Probe catalogue (datasheet values) and the derived envelope limits. . . . 7
3 Readers and how the input is recognised. . . . 7
4 Keywords that select columns (case ignored). . . . 8
5 Command-line options (from the program's parser). . . . 13
6 Output files (folder <stem>_fft). . . . 14
7 Pages written for every time series (prefix ch<k>_ for spectrogram channels). . . . 15
8 Overview pages of a spectrogram (SignalShark, EHP-50 TS), before the channel pages. 15
9 Menus of the live window. . . . 20
10 Spyder cells of emf_envelope_fft_v0_0_2.py. . . . 22
11 Self-test result (program 0.0.2, run during this build). . . . 22
12 Troubleshooting. . . . 23
13 Python modules used by the analyzer. . . . 25
14 NumPy calls. . . . 25
15 SciPy calls. . . . 26
16 pandas calls. . . . 26
17 h5py calls. . . . 27
18 vszh5_writer calls. . . . 27
19 Qt.py, PySide6 and pyqtgraph calls. . . . 28
20 CuPy calls (only with –gpu). . . . 28
21 pypdf and Veusz (through vszh5_writer). . . . 29
22 Python standard library modules used. . . . 29
23 Revision history of the program, the writer module and this manual. . . . 29

List of Figures

1 Synthetic FieldMan/Narda-TSX log (EFD-4091): field against the instrument clock on a date axis. The 6 s gap at 300 s is deliberate. *(Synthetic data.)* . . . 16
2 Envelope amplitude spectrum of the same log: the 12 rpm radar family (0.2 Hz and harmonics), the 0.7 Hz source, the probe-corrected trace (dashed) and the X/Y/Z axes (dotted). *(Synthetic data.)* . . . 17
3 Chirp-Z zoom around the strongest line: the dense CZT samples (line) between the FFT bins (markers). *(Synthetic data.)* . . . 17
4 Probe integration model $|H(\nu)| = |\mathrm{sinc}(\nu T)|$ for $T = 265$ ms up to the Nyquist frequency. *(Synthetic data.)* . . . 18
5 Synthetic SignalShark spectrogram: two carriers whose levels vary at 3 Hz and 0.5 Hz. *(Synthetic data.)* . . . 18
6 Envelope spectrum of every RF bin of that spectrogram: each carrier shows its own modulation frequency. *(Synthetic data.)* . . . 19
7 Live window (View → Dark) following a growing file with 2 Hz and 7 Hz envelope tones. *(Synthetic data.)* . . . 21

1 Introduction

A broadband diode probe such as the Narda EFD-4091, EFD-0691, EF 0691 or EFD-9091 adds the power of every source in its frequency range and reports one number per sample [1, 2, 3]. Its log therefore holds no information about the radio frequency of a source. It does hold how the total field changes in time: a radar antenna rotating, a beam being scanned, a transmitter keyed on and off, a duty cycle. The *EMF Envelope FFT Analyzer* takes the Fourier transform of that envelope, the *modulation spectrum*, so that sources which share an RF band but change at different rates can be told apart and their rates measured (a rotating antenna appears as a line at its rotation frequency, with harmonics from the beam shape).

The program reads the files written by the software of the instruments in use: FieldMan and Narda-TSX exports for the EFD probes [5, 9], NBM-550 or NBM-TS logs for the EF 0691 probe [6], EHP-50 TS spectra of the EHP-50F low-frequency analyzer [7], and SignalShark spectrum or spectrogram files [8]. A SignalShark spectrogram, unlike a probe log, does resolve RF frequency; for it the envelope transform is taken for every frequency bin.

1.1 What the analysis can and cannot tell you

- **Can**: the rates at which the total field changes (rotation in rpm, scan or keying periods), the relative strength of each modulation, and which modulations come from the same source (harmonic families).
- **Cannot**: the RF frequency of a source measured with a diode probe; content faster than half the probe's sample rate (it folds back, Section 5.4); an exposure assessment, for which IEC 62232 [18] and calibrated probes (IEEE 1309 [17]) remain the reference.
- **Only partly**: content near the Nyquist frequency is attenuated by the probe's own averaging; the program models and can partly undo that (Section 5.11).

1.2 Status of this version

Version 0.0.2 is an internal draft. No real export of FieldMan, Narda-TSX, NBM-TS, EHP-50 TS or SignalShark has been available yet, so the readers recognise file layouts by heuristics (Section 4) and were tested on synthetic files written in the expected styles. Use `--inspect` on every new kind of file before trusting an analysis, and see Section 12 for the known limitations.

1.3 Conventions

- f is an RF (or spectrum) frequency, ν an envelope (modulation) frequency; f_s is the sample rate of the log, N the number of samples, and T the probe's integration time.
- $\operatorname{sinc} x = \sin(\pi x)/(\pi x)$, the normalised form used by NumPy.
- Versions follow Semantic Versioning (`X.Y.Z`); the manual carries the version of the program it documents.
- Commands are shown for a Windows or Linux shell inside the mamba environment; `python` is the environment's interpreter.

2 Installation

2.1 Mamba environment

The environment file creates an environment named `emf-fft` from conda-forge with Python 3.12 and every dependency [36]:

```
mamba env create -f emf_envelope_fft.yml
mamba activate emf-fft
python emf_envelope_fft_v0_0_2.py --selftest
```

The self-test (Section 11) must end with `SELF-TEST PASSED`. For a GPU (NVIDIA CUDA only), also install CuPy (`mamba install -c conda-forge cupy`) and add `--gpu`.

2.2 Package files

Keep the files together as delivered; `vszh5_writer.py` must sit in the same folder as the script, which puts its own folder first on the module search path so that this copy is used rather than an older one (e.g. from the QuickSyn package). Table 1 lists the package.

Table 1: Files of the package.

File	Type	Size	Description
`emf_envelope_fft_v0_0_2.py`	Python 3.12 script	3031 lines	The analyzer: readers, estimators, Veusz/CSV/JSON output, batch, live mode, self-test. Spyder cells (Table 10).
`vszh5_writer.py`	Python module	1209 lines	The author's Veusz HDF5 writer, 0.0.11: typed datasets, 2-D images, plot-page helpers, rendering to PDF (with bookmarks) and PNG. Must stay next to the script.
`emf_envelope_fft.yml`	mamba environment	–	Creates the environment emf-fft from conda-forge (Python 3.12 and every dependency; CuPy commented out).
`LICENSE`	text	–	GNU General Public License v3.0.
`docs/EMF_Envelope_FFT_Manual_v0.0.2.pdf`	PDF	–	This manual, with heading-based bookmarks.
`docs/EMF_Envelope_FFT_Manual_v0.0.2.tex`	LuaLaTeX source	–	Source of this manual; written by the build tool, compiles on its own with lualatex.
`docs/figures/*.png`	PNG	–	Figures of this manual: Veusz pages rendered from the self-test's synthetic files, and screenshots of the live window.
`tools/build_manual_pdf.py`	Python 3.12 script	1948 lines	Builds this manual (generated tables, self-test run, LuaLaTeX, bookmark check).

2.3 Veusz and LuaLaTeX (optional)

The `.vszh5` documents are written without Veusz; Veusz is needed only to open them or to render them with `--render` [23]. The renderer finds Veusz on the PATH, in the usual install folders, or through the environment variable `VEUSZ_EXE` (or `QUICKSYN_VEUSZ`), e.g. `set VEUSZ_EXE=C:\Program Files\Veusz\veusz.exe`. Rebuilding this manual needs `lualatex` from TeX Live or MiKTeX (or `EMF_FFT_LUALATEX` pointing to it); its headings become the PDF bookmarks through hyperref and bookmark [37].

2.4 Spyder

Select the interpreter of the `emf-fft` environment in Spyder (*Tools → Preferences → Python interpreter*; the environment includes `spyder-kernels`) [35]. The script is divided into `# %%` cells (Section 10); run them in order with *Run cell*, then use the scratch cell at the end.

3 Quick start

```
:: 1  check the installation
python emf_envelope_fft_v0_0_2.py --selftest

:: 2  see how a file will be read (columns, units, time base)
python emf_envelope_fft_v0_0_2.py --inspect export.csv

:: 3  analyse one probe log; also render PDF and PNG with Veusz
python emf_envelope_fft_v0_0_2.py export.csv --probe EFD-4091 --render

:: 4  undo the probe's averaging and force Lomb-Scargle
python emf_envelope_fft_v0_0_2.py export.csv --probe EFD-4091 --correct-probe --lomb on

:: 5  several files in parallel, on the GPU if one is available
python emf_envelope_fft_v0_0_2.py a.csv b.csv c.csv --jobs 3 --gpu

:: 6  SignalShark spectrogram, the five strongest carriers as channels
python emf_envelope_fft_v0_0_2.py rec.h5 --source signalshark --rf-channels auto:5

:: 7  EHP-50 TS: the whole folder of auto-saved spectra
python emf_envelope_fft_v0_0_2.py D:\EHP50\2026 --source ehp

:: 8  follow a file that another program is writing (live window)
python emf_envelope_fft_v0_0_2.py --live growing.csv --probe EF0691 --window-s 120
```

3.1 Typical workflow

1. Export the measurement from the instrument software as text (CSV/TXT) or Excel, or copy the SignalShark `.h5` file.
2. Run `--inspect` and check the detected delimiter, decimal mark, columns, unit and sample interval. Correct anything wrong with the override options (Section 4.6).
3. Run the analysis with `--probe` set, so that the sample rate, integration time and range limits of the probe are known.
4. Open `<stem>_envelope_fft.vszh5` in Veusz, or use the rendered PDF, whose bookmarks list every page (Section 7).
5. Read `<stem>_summary.json`: peaks, harmonic families with their rpm, and warnings about timing, gaps, aliasing and range.

4 Instruments and input files

4.1 Probe catalogue

Table 2 lists the probes the program knows, from the datasheets. `--probe` accepts the names in the first column (case and spaces ignored) and the aliases `EFD-4091E` (treated as the EFD-4091; no separate datasheet for an “E” variant was found), `EF-0691`, `EFD-1891FAST` and the forms without hyphens. The last four columns are derived: the Nyquist frequency $f_s/2$, the response of the probe’s averaging at that frequency, and the first null $1/T$ of that response (Section 5.4).

Table 2: Probe catalogue (datasheet values) and the derived envelope limits.

Probe	RF range	f_s (Hz)	T (ms)	ν_N (Hz)	$\lvert H(\nu_N)\rvert$ (dB)	$1/T$ (Hz)	True RMS to (V/m)	Axes	Ref.
EFD-4091 2462/19	2 MHz -- 40 GHz	5	265	2.5	-7.6	3.774	65	rss	[1]
EFD-0691 2462/14B	100 kHz -- 6 GHz	5	265	2.5	-7.6	3.774	17	xyz	[2]
EF 0691 (NBM) NBM probe	100 kHz -- 6 GHz	5	n/p (1/fs used)	2.5	-3.9	5	22	rss	[6]
EFD-9091 2462/18	100 MHz -- 90 GHz	5	275	2.5	-8.3	3.636	61.4	rss	[3]
EFD-1891 Fast 2462/23	2 MHz -- 18 GHz	900	2.3	450	-29.4	434.8	–	rss	[4]

4.2 Readers

The reader is chosen from the file (Table 3); `--source` forces one.

Table 3: Readers and how the input is recognised.

`--source`	Instrument / software	Files	Auto-detected when	Result
`fieldman`, `tsx`	FieldMan or Narda-TSX with EFD-4091, -0691, -9091, -1891	.csv .txt	default for text files	time series (+ X/Y/Z)
`nbm`	NBM-550 / NBM-TS with EF 0691	.csv .txt	a column named 'Result ...'	time series
`xlsx`	any of the above saved as Excel	.xlsx .xls	by extension	time series
`ehp`	EHP-50 TS with EHP-50F (1 Hz - 400 kHz)	folder, `hh_mm_ss_cc.TXT`	a folder, or the EHP file name	spectrogram (folder), spectrum, or time series (logger file)
`signalshark`	SignalShark	.h5 .hdf5; converter .csv	.h5 extension; 8+ numeric column names or a frequency column	spectrogram (or level time series)

4.3 How text exports are read

Narda's software is German-made, so semicolon-separated files with a decimal comma are expected as often as comma-separated files with a decimal point.

1. **Encoding**: UTF-8 (with or without byte-order mark), then Windows-1252, then Latin-1.
2. **Delimiter**: the first of tab, `;`, `,`, `|` that occurs the same number of times on the last 50 non-blank lines; whitespace otherwise. Semicolon is tried before comma because a decimal comma puts a comma inside every number.
3. **Decimal mark**: comma when numbers such as `1,23` appear and the delimiter is not a comma.
4. **Header row**: the first row with the same number of fields as the row below it and more text fields than that row, whose fields are mostly numbers or date/times. Lines above it are kept as metadata and searched for a unit.

5. **Columns**: chosen with the keyword lists of Table 4; the unit comes from `--unit`, the value column's header or the metadata, in that order.
6. **Time**: numbers are seconds (milliseconds when the header says `ms`); anything else is parsed as date/time, day first when dates look like `dd.mm.yyyy`. A date column and a clock column are joined. The first value is the origin of every chunk of a large file, so the time axis is continuous.
7. **Size**: the file is read in chunks of 500 000 rows; columns longer than 20 000 000 samples move to a temporary file on disk (Section 9).

Table 4: Keywords that select columns (case ignored).

Use	Keywords	Note
time column	`time`, `zeit`, `uhrzeit`, `timestamp`, `date`, `datum`, `elapsed`, `t [s]`, `t(s)`, `t/s`, `sec`, `hh:mm`	a date column and a clock column are joined; otherwise the first match
value column, preferred (+5)	`actual`, `act`, `rss`, `total`, `result 1`, `isotrop`, `value`, `wert`, `e-field`, `e field`, `field`, `level`	+3 more if the header names a unit; leftmost column wins ties
value column, avoided (-6)	`max`, `min`, `avg`, `mean`, `average`, `batt`, `temp`, `humid`, `press`, `lat`, `lon`, `alt`, `gps`, `index`, `no.`, `nr`, `flag`, `dist`, `comment`	statistics and house-keeping columns
axis columns	`X`, `Y`, `Z`, `E_x`, `H y`, `field_z`, ...	used for per-axis spectra; their RSS is the value when no value column exists

4.4 EHP-50 TS spectra

EHP-50 TS saves one spectrum per file as `YYYY/MM/DD/hh_mm_ss_cc.TXT` (cc = hundredths of a second) [7]. Given a folder, the program reads every `.txt`/`.csv` below it, takes the time of each spectrum from its path (the file date when the name does not match), sorts them, and puts every spectrum on the frequency axis of the first one (linear interpolation when they differ). The total or RSS column is preferred over X, Y, Z. The result is analysed as a spectrogram (Section 5.14). Note that EHP-50 TS records only the EHP-50F (1 Hz to 400 kHz); it cannot record the RF probes.

4.5 SignalShark files

The layout of SignalShark's `.h5` files is not published. The reader takes the largest two-dimensional numeric dataset as levels (time × frequency, transposed if the axes say so), a one-dimensional dataset of matching length whose name or values look like frequencies as the frequency axis, and one that increases as the time axis; otherwise start/stop/centre/span and time-resolution attributes. `--h5-level`, `--h5-freq` and `--h5-time` name the datasets explicitly; `--inspect` prints the whole HDF5 tree. CSV files made with Narda's NSL *Converter* script [8] are read in wide format (one row per spectrum, the header holding the frequencies), long format (time, frequency, level columns) or transposed wide format (one row per frequency). Levels in dBm or dBµV are converted to field strength only when an antenna factor is given (`--antenna-factor-db`).

4.6 Overriding the detection

`--source`, `--time-col` (`"Date+Time"` joins two columns), `--value-col`, `--unit`, `--sep`, `--decimal`, `--dayfirst` and `--fs` (for files without a time column) override the corresponding guess. Column names or zero-based indices are accepted.

5 Theory and methods

5.1 Square-law detection adds powers

A diode detector operated in its square-law region delivers a voltage proportional to the square of the RF voltage. Let the field at the probe be the sum of M sources,

$$E(t) = \sum_{i=1}^{M} E_i \cos(2\pi f_i t + \varphi_i). \tag{5.1}$$

The detector output, averaged by a video filter that is much slower than any difference frequency $|f_i - f_j|$, is

$$\begin{aligned} \overline{E^2} &= \sum_i \sum_j E_i E_j \, \overline{\cos(2\pi f_i t + \varphi_i)\cos(2\pi f_j t + \varphi_j)} \\ &= \sum_i \sum_j \frac{E_i E_j}{2}\left[\overline{\cos\left(2\pi(f_i - f_j)t + \varphi_i - \varphi_j\right)} + \overline{\cos\left(2\pi(f_i + f_j)t + \varphi_i + \varphi_j\right)}\right] \\ &= \sum_i \frac{E_i^2}{2} = \sum_i E_{i,\mathrm{rms}}^2, \end{aligned} \tag{5.2}$$

because every term with $i \neq j$ and every sum-frequency term averages to zero. Three orthogonal sensors add in the same way, $E^2 = E_x^2 + E_y^2 + E_z^2$ (the isotropic RSS value). With the probe's frequency response $r(f)$ and the plane-wave power density $S = E_{\mathrm{rms}}^2 / Z_0$, the displayed value is

$$S_{\mathrm{disp}}(t) = \sum_i r(f_i)\, S_i(t), \qquad E_{\mathrm{disp}}(t) = \sqrt{Z_0\, S_{\mathrm{disp}}(t)}. \tag{5.3}$$

Equation (5.2) holds only while every diode stays in its square-law region, which for the EFD probes is the True-RMS range of Table 2; above it the program warns.

5.2 Why the transform is taken of power

The Fourier transform is linear, so applying it to (5.3) gives

$$\mathcal{F}\{S_{\mathrm{disp}}\}(\nu) = \sum_i r(f_i)\, \mathcal{F}\{S_i\}(\nu)\,: \tag{5.4}$$

the envelope spectrum of the log is the weighted sum of the envelope spectra of the sources, and each source's modulation lines appear unchanged. The field $E_{\mathrm{disp}} = \sqrt{Z_0 S_{\mathrm{disp}}}$ does not have this property. Take two sources $S_1 = a + b\cos 2\pi\nu_1 t$ and $S_2 = c + d\cos 2\pi\nu_2 t$ with $A = a + c$ and $x(t) = b\cos 2\pi\nu_1 t + d\cos 2\pi\nu_2 t$, $|x| < A$. The Taylor series of the square root gives

$$\sqrt{A + x} = \sqrt{A}\left(1 + \frac{x}{2A} - \frac{x^2}{8A^2} + \ldots\right), \tag{5.5}$$

$$\begin{aligned} x^2 = {} & \frac{b^2 + d^2}{2} + \frac{b^2}{2}\cos 4\pi\nu_1 t + \frac{d^2}{2}\cos 4\pi\nu_2 t \\ & + bd\left[\cos 2\pi(\nu_1 - \nu_2)t + \cos 2\pi(\nu_1 + \nu_2)t\right], \end{aligned} \tag{5.6}$$

so the spectrum of E contains lines at $2\nu_1$, $2\nu_2$ and $\nu_1 \pm \nu_2$ that no source produces (intermodulation). The program therefore analyses power density (W/m^2) by default (`--quantity power`); `field` and `db` are available for comparison only.

5.3 Unit conversions

- Power density from electric field (far field): $S = E^2/Z_0$ with $Z_0 = \mu_0 c$. The program uses $Z_0 = 376.730313668\,\Omega$ (from the CODATA 2018 μ_0); the CODATA 2022 value is $376.730\,313\,412(59)\,\Omega$ [32], a difference of 7×10^{-10}. Narda's displays use round conversions near $377\,\Omega$; `--z0 377` reproduces them.
- From magnetic field: $S = Z_0 H^2$ (far field only). Magnetic flux density is analysed as B^2 (proportional to the magnetic energy density), and EHP-50F fields as the field squared, because at 1 Hz–400 kHz the fields are quasi-static and no plane-wave power density exists.
- $1\,\mathrm{mW/cm^2} = 10^{-3}\,\mathrm{W}/10^{-4}\,\mathrm{m^2} = 10\,\mathrm{W/m^2}$; $1\,\mu\mathrm{W/cm^2} = 10^{-2}\,\mathrm{W/m^2}$.
- From dBµV/m: $E = 10^{(L-120)/20}\,\mathrm{V/m}$, because $L = 20\log_{10}(E/1\,\mu\mathrm{V/m})$ and $1\,\mu\mathrm{V} = 10^{-6}\,\mathrm{V}$.
- From dBm at a 50 Ω port: $P = V^2/R$ gives

$$\begin{aligned} L_{\mathrm{dB\mu V}} &= 20\log_{10}\frac{V}{1\,\mu\mathrm{V}} = 10\log_{10}\frac{PR}{10^{-12}\,\mathrm{V^2}} \\ &= 10\log_{10}\frac{P}{1\,\mathrm{mW}} + 10\log_{10}\frac{10^{-3}\cdot 50}{10^{-12}} = L_{\mathrm{dBm}} + 106.9897, \end{aligned} \tag{5.7}$$

  rounded to 107 dB, and with the antenna factor $AF = E/V$ in dB(1/m), $L_{\mathrm{dB\mu V/m}} = L_{\mathrm{dB\mu V}} + AF$. Without `--antenna-factor-db`, dBm levels are analysed as mW at the analyzer port.

5.4 Sampling, probe averaging and aliasing

A log sampled at f_s holds envelope content up to the Nyquist frequency $\nu_N = f_s/2$, with resolution $\Delta\nu = f_s/N = 1/T_{\mathrm{rec}}$. Each sample is an average over the integration time T. Modelling that average as a moving (boxcar) average,

$$y(t) = \frac{1}{T}\int_{t-T}^{t} x(\tau)\,d\tau = (h * x)(t), \qquad h(t) = \begin{cases} 1/T, & 0 \le t < T \\ 0, & \text{otherwise,} \end{cases} \tag{5.8}$$

its frequency response is

$$\begin{aligned} H(\nu) &= \int_0^T \frac{1}{T} e^{-i2\pi\nu t}\,dt = \frac{1-e^{-i2\pi\nu T}}{i2\pi\nu T} = e^{-i\pi\nu T}\,\frac{e^{i\pi\nu T} - e^{-i\pi\nu T}}{2i\,\pi\nu T} \\ &= e^{-i\pi\nu T}\,\frac{\sin\pi\nu T}{\pi\nu T} = e^{-i\pi\nu T}\,\mathrm{sinc}(\nu T), \qquad |H(\nu)| = |\mathrm{sinc}(\nu T)|. \end{aligned} \tag{5.9}$$

For the EFD-4091 ($T = 265$ ms) this gives -2.39 dB at 1.5 Hz and -7.55 dB at the Nyquist frequency 2.5 Hz, with the first null at $1/T = 3.774$ Hz. Sampling folds any content at ν to

$$\nu_{\mathrm{alias}} = \left|\nu - k f_s\right|, \qquad k = \mathrm{round}(\nu/f_s), \tag{5.10}$$

attenuated only by $|H(\nu)|$: a 3 Hz envelope line at $f_s = 5$ Hz appears at 2 Hz, -12.4 dB down. The boxcar is a model; Narda publishes only a nominal integration time. Figure 4 shows the model, and the program reports in the warnings whether the first null lies above or below ν_N.

5.5 Putting the log on a uniform grid

The FFT needs equally spaced samples. The sample interval is the mean of the intervals that are not gaps (an interval longer than 1.6 times the median is a gap). The mean is used rather than the median because time stamps rounded to a microsecond make the median of a 900 Hz log 0.001111 s, and a grid 0.01 % off drifts in phase against the data, which adds side lines and loses amplitude. When the stamps are within 1 % of uniform (median $|\Delta t - \overline{\Delta t}|/\overline{\Delta t} < 0.01$), with no gaps and the expected count, the samples are used as they are; otherwise they are linearly interpolated onto the grid, gaps bridged. The gap-free blocks are recorded for Welch, and the raw stamps are kept for Lomb–Scargle.

5.6 Amplitude spectrum

With a window $w[n]$ and the mean removed, the DFT of the record is

$$X[k] = \sum_{n=0}^{N-1} w[n]\,(x[n] - \bar{x})\,e^{-i2\pi kn/N}, \qquad \nu_k = k\,\frac{f_s}{N}. \tag{5.11}$$

For a sinusoid $x[n] = A\cos(2\pi k_0 n/N + \varphi)$ on bin k_0,

$$X[k_0] = \frac{A}{2}e^{i\varphi}\sum_n w[n] + \frac{A}{2}e^{-i\varphi}\sum_n w[n]\,e^{-i4\pi k_0 n/N} \approx \frac{A}{2}e^{i\varphi}\sum_n w[n], \tag{5.12}$$

because the second sum is the window's transform $2k_0$ bins away from its centre. Hence the single-sided amplitude

$$\hat{A}_k = \frac{2\,|X[k]|}{\sum_n w[n]} \quad (k \neq 0,\ k \neq N/2), \tag{5.13}$$

and without the factor 2 at DC and Nyquist. $\sum_n w[n]/N$ is the window's coherent gain (0.5 for Hann) [16]. A line between two bins reads low by the scalloping loss, −1.42 dB for Hann at half a bin (computed by the build tool); the chirp-Z zoom removes it (Section 5.8). Zero padding interpolates the same spectrum; it does not improve the resolution f_s/N.

5.7 Welch power spectral density

The record is cut into K segments of L samples (`--nperseg`; by default the largest power of two giving about seven averages) with 50 % overlap, each windowed and transformed [11]:

$$\hat{P}(\nu_k) = \frac{c_k}{K f_s \sum_n w[n]^2}\sum_{m=0}^{K-1}\left|\sum_{n=0}^{L-1} w[n](x_m[n] - \bar{x}_m)e^{-i2\pi kn/L}\right|^2, \qquad c_k = \begin{cases}1 & k = 0,\ L/2\\ 2 & \text{otherwise.}\end{cases} \tag{5.14}$$

The normalisation follows from white noise of variance σ^2: then $\mathrm{E}|X_m[k]|^2 = \sigma^2\sum_n w[n]^2$, so $\hat{P} = 2\sigma^2/f_s$ and $\int_0^{f_s/2}\hat{P}\,d\nu = \sigma^2$ (Parseval). Averaging K segments reduces the variance of the estimate roughly K-fold at the price of resolution f_s/L. The program takes segments only inside gap-free blocks and processes them 512 at a time, so the record may be a disk-mapped array; the result equals `scipy.signal.welch` to rounding error (Table 11).

5.8 Chirp-Z zoom

The chirp-Z transform evaluates

$$X_k = \sum_{n=0}^{N-1} x[n]\,z_k^{-n}, \qquad z_k = A\,W^{-k}, \quad k = 0, \ldots, M-1, \tag{5.15}$$

on an arc of the z plane [12]. For a zoom from ν_1 to ν_2 on the unit circle, $A = e^{i2\pi\nu_1/f_s}$ and $W = e^{-i2\pi\Delta\nu/f_s}$ with $\Delta\nu = (\nu_2 - \nu_1)/(M-1)$. Bluestein's identity $nk = \frac{1}{2}[n^2 + k^2 - (k-n)^2]$ [19] turns the sum into a convolution,

$$X_k = W^{k^2/2}\sum_{n=0}^{N-1}\left(x[n]\,A^{-n}W^{n^2/2}\right)W^{-(k-n)^2/2}, \tag{5.16}$$

which FFTs evaluate in $O((N+M)\log(N+M))$ operations (`scipy.signal.zoom_fft` [31]). The program zooms ±16 FFT bins around the strongest line with 4096 points by default (1/128 of a bin). The zoom samples the same windowed spectrum more densely: it locates a peak and reads its amplitude without scalloping loss, but it does not separate two lines closer than f_s/N. Figure 3 shows the dense samples between the FFT bins.

5.9 Lomb–Scargle periodogram

For irregular stamps t_j (jitter, gaps), the classical Lomb–Scargle periodogram fits a sinusoid at each frequency by least squares without resampling [13, 14]. With $\omega = 2\pi\nu$, the offset τ is defined by

$$\tan 2\omega\tau = \frac{\sum_j \sin 2\omega t_j}{\sum_j \cos 2\omega t_j}, \tag{5.17}$$

which makes $\sum_j \cos\omega(t_j-\tau)\sin\omega(t_j-\tau) = 0$, and

$$P(\omega) = \frac{1}{2}\left\{\frac{\left[\sum_j y_j \cos\omega(t_j-\tau)\right]^2}{\sum_j \cos^2\omega(t_j-\tau)} + \frac{\left[\sum_j y_j \sin\omega(t_j-\tau)\right]^2}{\sum_j \sin^2\omega(t_j-\tau)}\right\}. \tag{5.18}$$

For $y_j = a\cos\omega(t_j-\tau) + b\sin\omega(t_j-\tau)$, $a^2+b^2=A^2$, the orthogonality gives $\sum_j y_j\cos(\cdot) = a\sum_j\cos^2(\cdot)$ and $\sum_j y_j\sin(\cdot) = b\sum_j\sin^2(\cdot)$, so $P = \frac{1}{2}(a^2\sum\cos^2 + b^2\sum\sin^2)$. With samples spread over many cycles $\sum\cos^2 \approx \sum\sin^2 \approx N/2$, hence

$$P \approx \frac{NA^2}{4} \quad\Longrightarrow\quad \hat{A} = \sqrt{\frac{4P}{N}}, \tag{5.19}$$

the amplitude the program reports, directly comparable with (5.13). It runs automatically when more than 0.1 % of the grid lies in gaps or the jitter exceeds 1 % (`--lomb auto`), in chunks of frequencies to bound memory.

5.10 Envelope spectrogram

For records of at least 16 Welch segments, the envelope is also shown as a short-time PSD (time × envelope frequency), computed with `scipy.signal.ShortTimeFFT` in one-sided, PSD-scaled form [30], the hop chosen so that at most 2000 slices are produced and slices hanging over the ends omitted. It shows whether a modulation is steady or comes and goes.

5.11 Probe correction

`--correct-probe` divides out the averaging (5.9) where that is safe. A plain inverse $1/|H|$ grows without bound near the null at $1/T$ and amplifies noise, so the gain is limited to $g_{\max}$ (`--max-boost-db`, default 12 dB, $g_{\max} = 3.981$) with a Wiener-type form:

$$G(h) = \frac{h}{h^2+\varepsilon}, \qquad h = |H(\nu)|. \tag{5.20}$$

Its maximum over h follows from $dG/dh = (\varepsilon - h^2)/(h^2+\varepsilon)^2 = 0$: $h = \sqrt{\varepsilon}$ and $G_{\max} = 1/(2\sqrt{\varepsilon})$. Setting $G_{\max} = g_{\max}$ gives

$$\varepsilon = \frac{1}{4g_{\max}^2} = 0.01577. \tag{5.21}$$

Where $h \geq 1/g_{\max}$ the exact inverse $1/h$ ($\leq g_{\max}$) is used, below it $G(h)$; bins at or beyond the first null ($\nu T \geq 1$) are left empty (NaN). At the switch point $h = 1/g_{\max}$ the two gains are $g_{\max}$ and $G = \frac{4}{5}g_{\max}$, a step of 1.94 dB (Section 12). For the 5 Hz probes $|H| > 1/g_{\max}$ up to the Nyquist frequency (−7.55 dB at 2.5 Hz), so the exact inverse applies throughout; the step can appear only for probes whose first null lies below their Nyquist frequency (EFD-1891 fast: 434.8 Hz against 450 Hz).

5.12 Peaks and their exact frequency

Peaks are local maxima standing at least `--snr` times (default 6) above the median spectrum level, with a prominence of (snr − 1) times the median, within `--dyn-range-db` (default 50 dB) of the strongest peak, at most 12, DC and the first bin excluded. Each peak frequency is refined by a parabola through the logarithms of the three bins around it, $\alpha = \ln \hat{A}_{k-1}$, $\beta = \ln \hat{A}_k$, $\gamma = \ln \hat{A}_{k+1}$. Fitting $p(x) = ax^2 + bx + c$ through $(-1, \alpha)$, $(0, \beta)$, $(1, \gamma)$ gives $c = \beta$, $b = (\gamma - \alpha)/2$, $a = (\alpha - 2\beta + \gamma)/2$, and the vertex

$$\delta = -\frac{b}{2a} = \frac{1}{2}\,\frac{\alpha - \gamma}{\alpha - 2\beta + \gamma}, \qquad \hat{\nu} = \nu_k + \delta\,\Delta\nu. \tag{5.22}$$

The parabola is exact for a Gaussian window and a close approximation for Hann. The CZT then gives the frequency and amplitude of the strongest line (`f_hz_czt`, `amp_czt` in the summary).

5.13 Harmonic families and rotation rate

A rotating antenna passes its beam over the probe once per turn, which gives a line at the rotation frequency ν_0 and harmonics $k\nu_0$ from the beam shape. The program groups peaks into families: for each candidate ν_0 (itself a peak above two bins), the members are the peaks within $\max(1.5\,\Delta\nu,\ 0.01\,\nu_0)$ of a whole multiple $k\nu_0$, $k \geq 1$; the family with the largest total amplitude is taken, its members removed, and the search repeated (three families at most). Each family is reported with ν_0, the period $1/\nu_0$ and the rate $60\,\nu_0$ in rpm. In Figure 2 the 12 rpm family (0.2 Hz) and the separate 0.7 Hz source are found this way.

5.14 Spectrograms: envelope spectrum per frequency bin

For a spectrogram $L(t_m, f_j)$ (SignalShark, EHP-50 TS folder), the levels are converted to linear power $S(t_m, f_j)$ (Section 5.3), put on a uniform time grid at the mean spectrum interval, and transformed along time for every bin j at once:

$$\hat{A}_j(\nu_k) = \frac{2}{\sum_m w[m]}\left|\sum_m w[m](S(t_m, f_j) - \bar{S}_j)e^{-i2\pi km/N_t}\right|. \tag{5.23}$$

The *modulation map* (Figure 6) shows this in dB relative to its maximum; the strongest envelope frequency of every bin is plotted separately. In addition, *channels* are analysed like probe logs, using the sum of linear power over a band: the whole span (`total`) and the strongest spectral peaks (±2 bins, `--rf-channels auto:N`) or bands given in Hz (`--rf-channels f1:f2,f3:f4`). Images larger than 1500 pixels per side are reduced by taking the maximum of neighbouring cells for the Veusz document; the full-resolution arrays go to the `.npz` file.

6 Command-line reference

Table 5 is generated from the program's own argument parser, so it always matches `python emf_envelope_fft_v0_0_2.py --help`.

Table 5: Command-line options (from the program's parser).

Option	Default	Description
`-h, --help`	–	show the help text and exit
`inputs ...`	–	files or EHP-50 TS folders
`--probe TEXT`	–	probe model (EFD-4091, EFD-0691, EF0691, EFD-9091, EFD-1891F); sets fs and T_int defaults
`--source TEXT`	`auto`	force a reader instead of auto-detection (choices: auto, fieldman, tsx, nbm, ehp, signalshark, xlsx)
`--fs FLOAT`	–	sample rate (Hz) when the file has no time column

Table 5 (continued)

Option	Default	Description
`--t-int FLOAT`	–	probe integration time (s), overrides the catalogue
`--quantity TEXT`	`power`	quantity transformed: power (linear in sources), field or dB (choices: power, field, db)
`--window TEXT`	`hann`	SciPy window name (hann, blackmanharris, flattop, ...)
`--nperseg INT`	–	Welch segment length (samples)
`--zoom TEXT`	–	CZT zoom band 'f1:f2' in Hz (default: around the strongest peak)
`--zoom-points INT`	`4096`	number of CZT points in the zoom band
`--lomb TEXT`	`auto`	Lomb-Scargle on the raw stamps (auto: with gaps or jitter) (choices: auto, on, off)
`--correct-probe`	–	undo the integration roll-off
`--max-boost-db FLOAT`	`12.0`	largest gain the correction may apply
`--rf-channels TEXT`	`auto:3`	spectrogram channels: 'auto:N' or 'f1:f2,f3:f4' (Hz)
`--antenna-factor-db FLOAT`	–	AF (dB/m) for dBm/dBuV spectrogram levels
`--z0 FLOAT`	`376.730313668`	wave impedance (ohm); 377 matches Narda displays
`--snr FLOAT`	`6.0`	peak threshold over the median level (linear)
`--dyn-range-db FLOAT`	`50.0`	ignore peaks more than this many dB below the strongest
`--out TEXT`	–	output folder (default <input>_fft)
`--jobs INT`	`1`	parallel processes for several inputs
`--gpu`	–	use CuPy/CUDA FFTs when available
`--render`	–	PDF/PNG via vszh5_writer.py + Veusz
`--pickle`	–	also offload the full result to .pkl
`--time-col TEXT`	–	time column name/index ('Date+Time' joins two)
`--value-col TEXT`	–	value column name/index
`--unit TEXT`	–	unit of the values, e.g. 'V/m', 'mW/cm2', 'dBuV/m'
`--sep TEXT`	–	delimiter (default: sniffed)
`--decimal TEXT`	–	decimal mark (default: sniffed)
`--dayfirst`	–	dates are day-first
`--h5-level TEXT`	–	SignalShark .h5: level dataset path
`--h5-freq TEXT`	–	SignalShark .h5: frequency dataset path
`--h5-time TEXT`	–	SignalShark .h5: time dataset path
`--inspect`	–	only report how each input is read
`--live FILE`	–	follow a growing file and update live
`--window-s FLOAT`	`120.0`	live: rolling window (s)
`--update FLOAT`	`1.0`	live: update interval (s)
`--live-seconds FLOAT`	–	live: stop after this many seconds
`--no-gui`	–	live: console output only
`--selftest DIR`	–	run the built-in self-test (optionally keep outputs in DIR)
`--version`	–	show program's version number and exit

7 Output files

Results go to the folder `<stem>_fft` beside the input, or to `--out`.

Table 6: Output files (folder `<stem>_fft`).

File	Content	Written
`<stem>_envelope_fft.vszh5`	Veusz document: all datasets and pages	always
`<name>_amplitude.csv`	f_hz, amplitude [, amplitude_corrected, amplitude_X/Y/Z]	per series / channel
`<name>_welch.csv`	f_hz, psd (unit^2/Hz)	per series / channel
`<name>_zoom.csv`	f_hz, amplitude (chirp-Z)	when a peak or --zoom
`<name>_lomb.csv`	f_hz, amplitude (Lomb-Scargle)	gaps/jitter or --lomb on

Table 6 (continued)

File	Content	Written
`<stem>_modulation.npz`	full-resolution spectrum, modulation image and dominant modulation per bin	spectrograms
`<stem>_summary.json`	peaks, families, warnings, settings	always
`<stem>_result.pkl`	the whole result object	--pickle
`<stem>_envelope_fft.pdf`	all pages, bookmarked	--render
`<stem>_envelope_fft_pNN_<page>.png`	one image per page (150 dpi)	--render
`<input>.live.f64`	live mode: every sample, float64 (t, value) pairs	--live

7.1 The Veusz document

The `.vszh5` file is Veusz's native HDF5 format, written directly by `vszh5_writer.py` (Appendix A.5): the datasets with their types and the plots as the Veusz command script, so the file opens with *File → Open* and every plot can be edited there. It is written to `<name>.part` and moved into place at the end, so an open copy is never half-written. Tables 7 and 8 list the pages; the page titles are also the PDF bookmark titles.

Table 7: Pages written for every time series (prefix `ch<k>_` for spectrogram channels).

Page	Bookmark title	Content	When
`time`	Field vs time	field strength against time (a date axis when the start time is known)	always
`amplitude`	Envelope amplitude spectrum (FFT)	single-sided amplitude spectrum, log scale; probe-corrected and X/Y/Z traces	always
`welch`	Welch power spectral density	Welch PSD in dB, number and length of segments in the title	always
`zoom`	Chirp-Z zoom around the strongest peak	chirp-Z samples (line) and FFT bins (markers) around the strongest line	a peak
`lomb`	Lomb-Scargle periodogram (raw stamps)	Lomb-Scargle amplitude from the raw stamps against the FFT	Lomb on
`probe`	Probe integration model |sinc|	|sinc(f T)| of the probe averaging in dB	T known
`spectrogram`	Envelope spectrogram	image: time x envelope frequency, PSD in dB (60 dB range)	long logs

Table 8: Overview pages of a spectrogram (SignalShark, EHP-50 TS), before the channel pages.

Page	Bookmark title	Content
`spectrum`	Mean and max-hold spectrum	mean and max-hold level per frequency (dB re W/m^2 or as given)
`spectrogram`	Spectrogram (level vs time and frequency)	image: frequency x time, level in dB
`modulation_map`	Envelope spectrum per frequency bin	image: frequency x envelope frequency, dB rel. maximum
`dominant_modulation`	Strongest envelope frequency per bin	strongest envelope frequency of every bin

7.2 Rendering to PDF and PNG

With `--render`, `vszh5_writer.render_pages` drives Veusz in listen mode and writes one multipage PDF and one PNG per page (`<stem>_envelope_fft_pNN_<page>.png`). The PDF gets a bookmark tree: the

pages of a single log as a flat list; for a spectrogram an *Overview* entry, then one entry per channel (*Channel 0: …*) holding its pages.

7.3 Summary file

`<stem>_summary.json` holds the input and quantity (`unit_in`, `quantity`, `quantity_note`), timing (`fs_hz`, `nyquist_hz`, `samples`, `duration_s`, `resolution_hz`, `start`, `t_int_s`), the Welch settings, `peaks` (`f_hz`, `amp`, `snr_db`, and `f_hz_czt`, `amp_czt` for the zoomed line), `harmonic_families` (`f0_hz`, `period_s`, `rpm`, `harmonics`), the `warnings`, the compute back end and the program version. For a spectrogram it holds the frequency range and bin count, the spectrum rate and one such block per channel.

7.4 Examples

Figures 1–6 were rendered by this program from the synthetic files of its self-test.

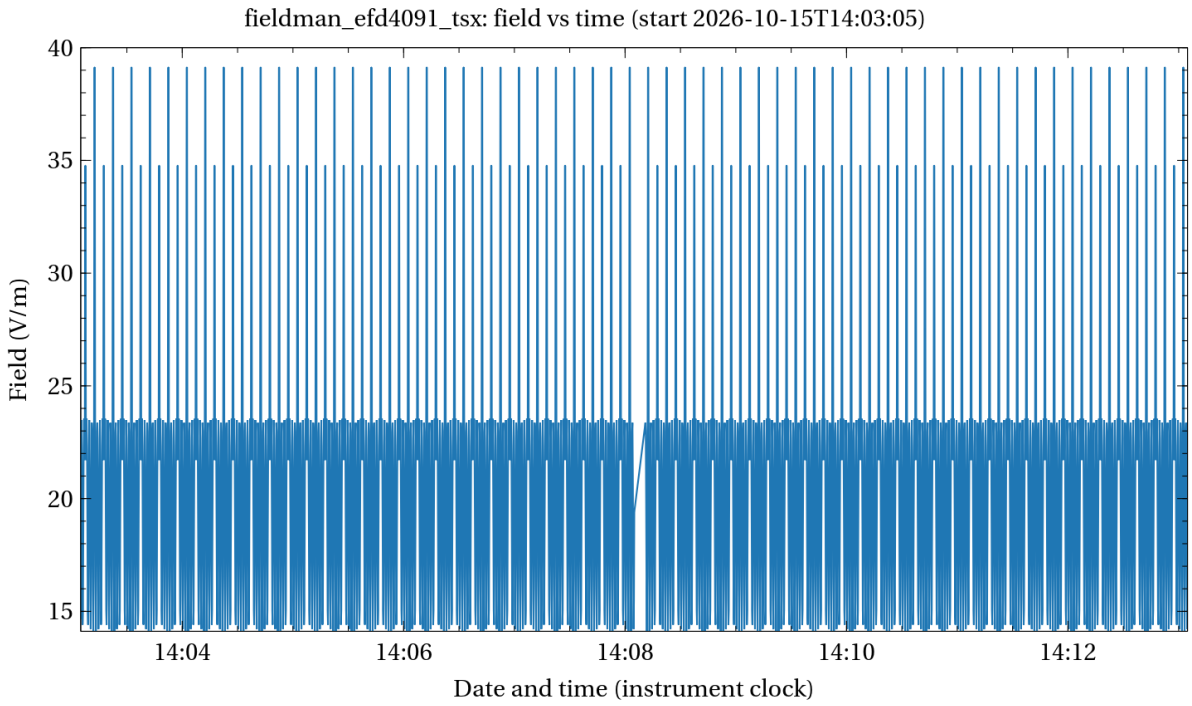


Figure 1: Synthetic FieldMan/Narda-TSX log (EFD-4091): field against the instrument clock on a date axis. The 6 s gap at 300 s is deliberate. *(Synthetic data.)*

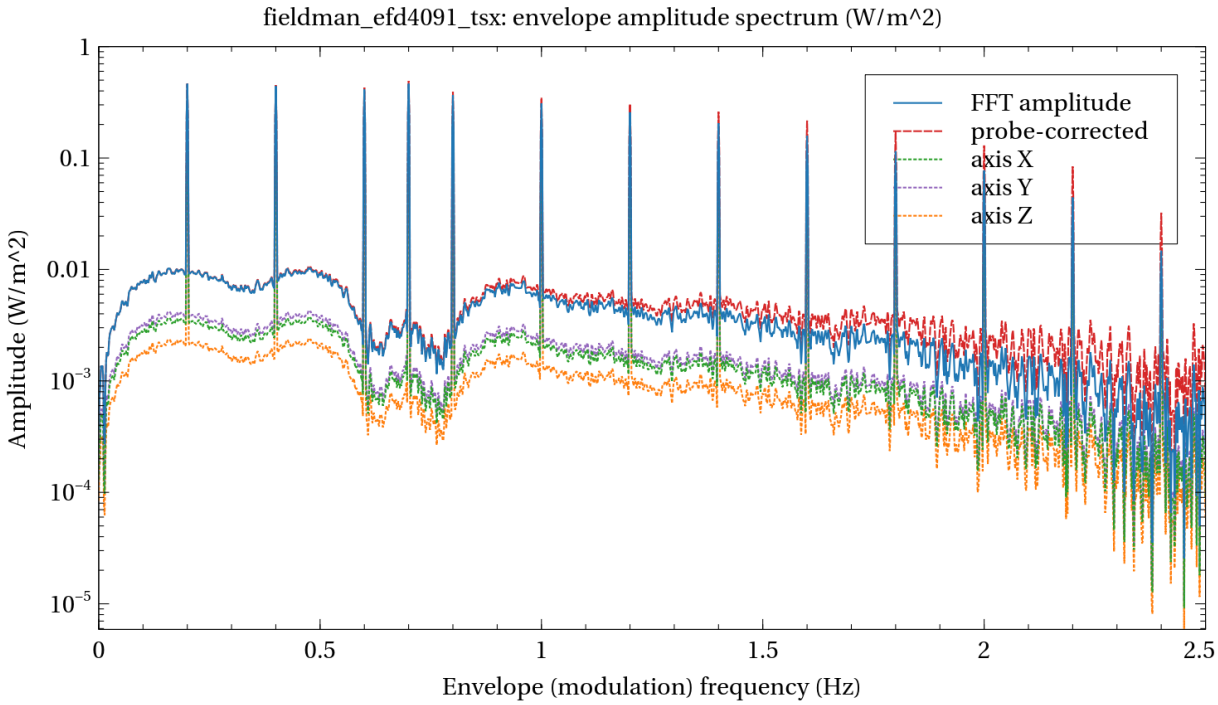


Figure 2: Envelope amplitude spectrum of the same log: the 12 rpm radar family (0.2 Hz and harmonics), the 0.7 Hz source, the probe-corrected trace (dashed) and the X/Y/Z axes (dotted). *(Synthetic data.)*

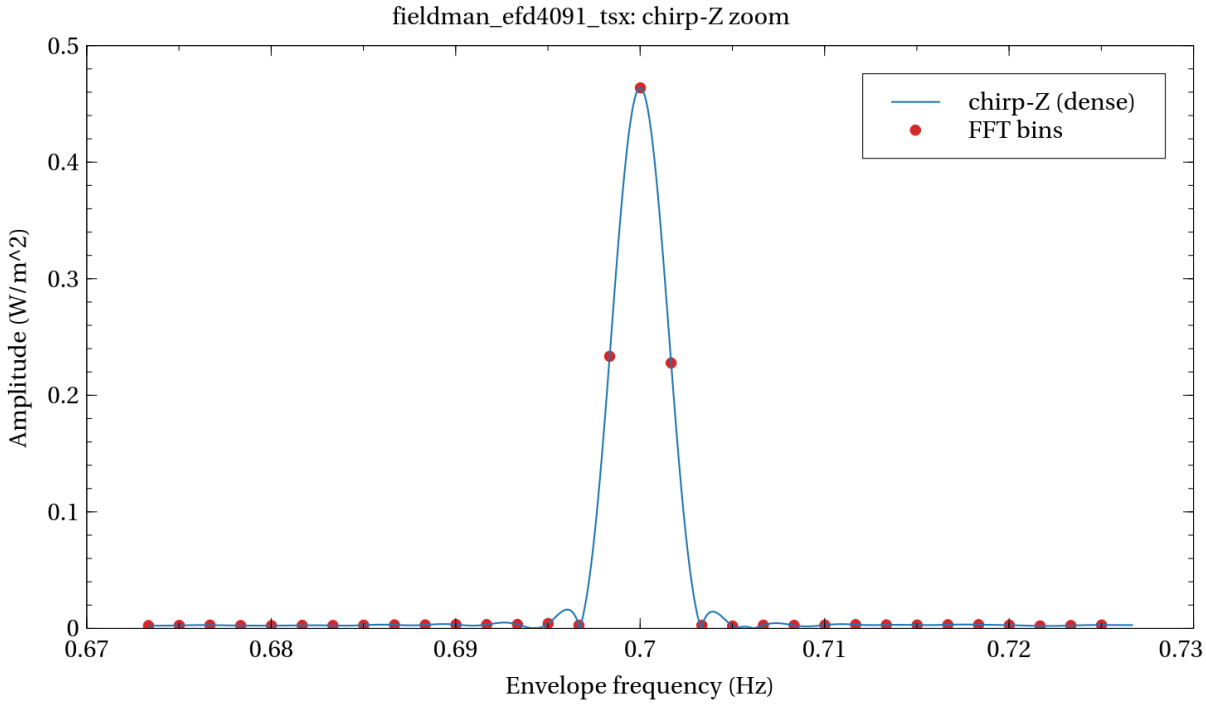


Figure 3: Chirp-Z zoom around the strongest line: the dense CZT samples (line) between the FFT bins (markers). *(Synthetic data.)*

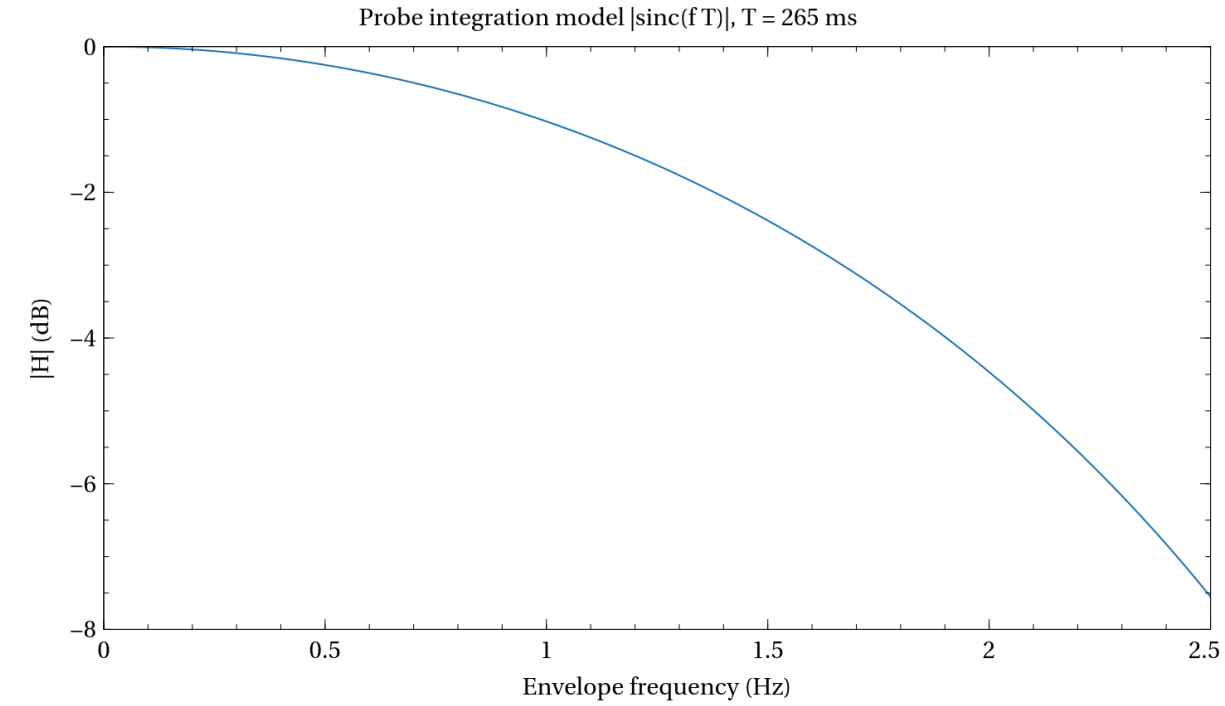


Figure 4: Probe integration model $|H(\nu)| = |\mathrm{sinc}(\nu T)|$ for $T = 265\,\mathrm{ms}$ up to the Nyquist frequency. *(Synthetic data.)*

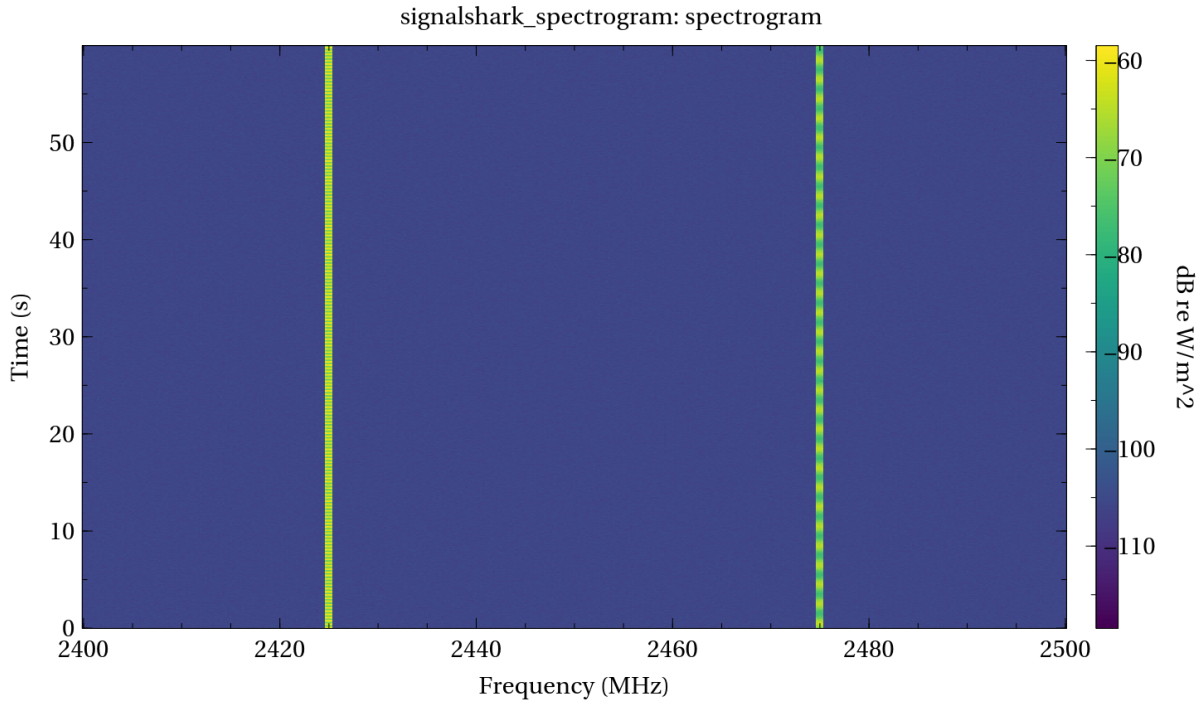


Figure 5: Synthetic SignalShark spectrogram: two carriers whose levels vary at 3 Hz and 0.5 Hz. *(Synthetic data.)*

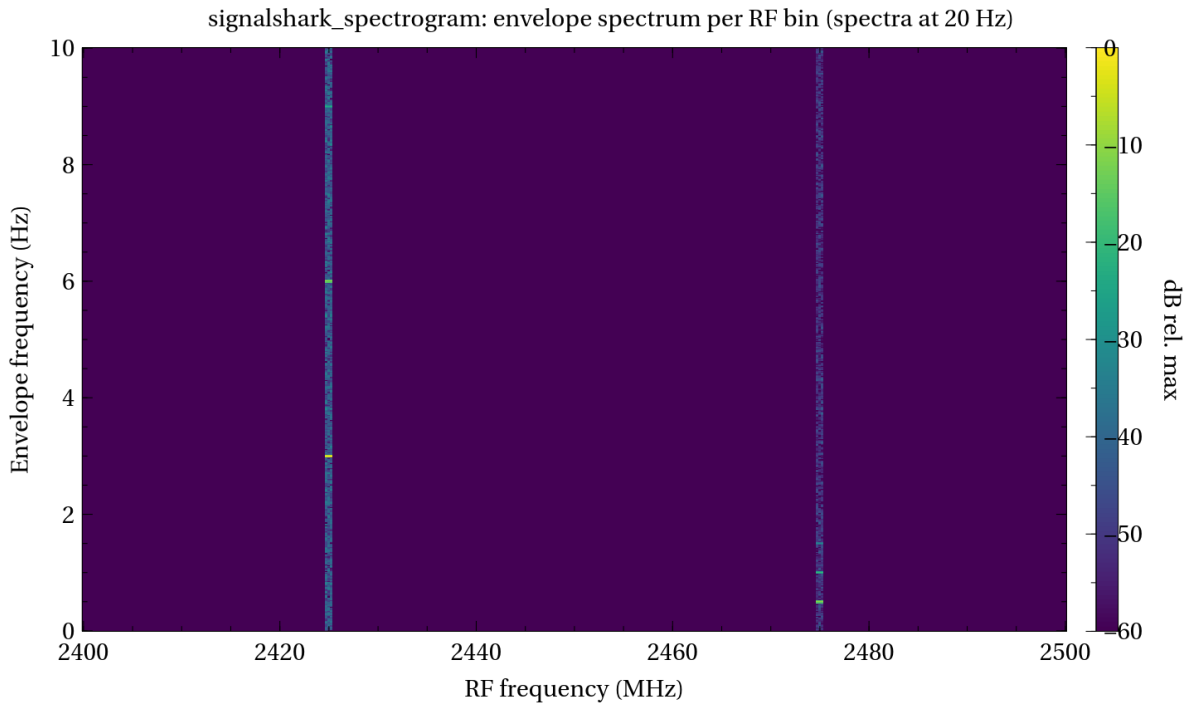


Figure 6: Envelope spectrum of every RF bin of that spectrogram: each carrier shows its own modulation frequency. *(Synthetic data.)*

8 Live mode

`--live FILE` follows a text file that another program appends to. New complete lines are read every `--update` seconds (default 1 s; an unfinished last line waits for the next poll), kept in a ring buffer twice the size of the analysis window `--window-s` (default 120 s), and the envelope amplitude spectrum and Welch PSD of the latest window are recomputed and drawn. Every sample is also appended to `<file>.live.f64`, raw float64 pairs (time, value) that can be read back with `np.fromfile(path).reshape(-1, 2)`. `--no-gui` prints the strongest lines and families instead; `--live-seconds` stops after a time.

Table 9: Menus of the live window.

Menu	Entry	Action
File	Save snapshot (.vszh5)	full analysis of the current window; writes the usual output files
File	Clear buffer (free RAM)	empties the ring buffer, runs flush_memory()
File	Quit	closes the window
View	Light / Dark	palette of the window and plot colours
Analysis	Correct probe response	applies --correct-probe live

Neither FieldMan nor Narda-TSX documents a continuously written file or a streaming interface for the broadband probes [5, 9], so whether live mode works with them depends on how they write data; it was tested with a file written in real time by a script. NBM-550 remote output (`MEAS_START`, 5, 50 or 60 samples per second [6]) logged to a file by another tool is a documented route.

9 Memory, parallel processing and GPU

- **Spill to disk**: a column longer than 20 000 000 samples is moved from RAM to a temporary raw file and used as a memory map; the files are deleted when the program exits. A 2.4-million-row, 900 Hz test log was analysed this way.
- **Chunks and batches**: text files are read 500 000 rows at a time; Welch processes 512 segments at a time; Lomb–Scargle about 64 MiB of frequencies at a time; the per-bin modulation spectra in batches of about 256 MiB.
- **Freeing memory**: `flush_memory()` runs the garbage collector and empties the CuPy memory pools; batch mode calls it after each file, and the live window's *File → Clear buffer* too. `--pickle` also writes the whole result object to `<stem>_result.pkl` (load it with `load_pickle`, only from files you trust).
- **Parallel**: `--jobs N` analyses N files in separate processes; each process then uses one FFT thread so that the cores are not oversubscribed. With one job, SciPy's FFTs use every core.
- **GPU**: `--gpu` moves the FFTs, Welch segments, modulation spectra and Lomb–Scargle sums to an NVIDIA GPU through CuPy [27]. Without CuPy or a CUDA device the program continues on the CPU and says why in the summary (`backend`).

10 Working in Spyder

The script is organised in Spyder cells, each opened by a banner `# %% ====` and a title line. Table 10 lists them with the functions and classes each defines (generated from the script). Run the cells from the top; the last cell is a commented scratch cell:

```
opts = Options(probe="EFD-4091", correct_probe=True, lomb="on")
print(inspect_input(r"C:\path\to\export.csv", opts))
```

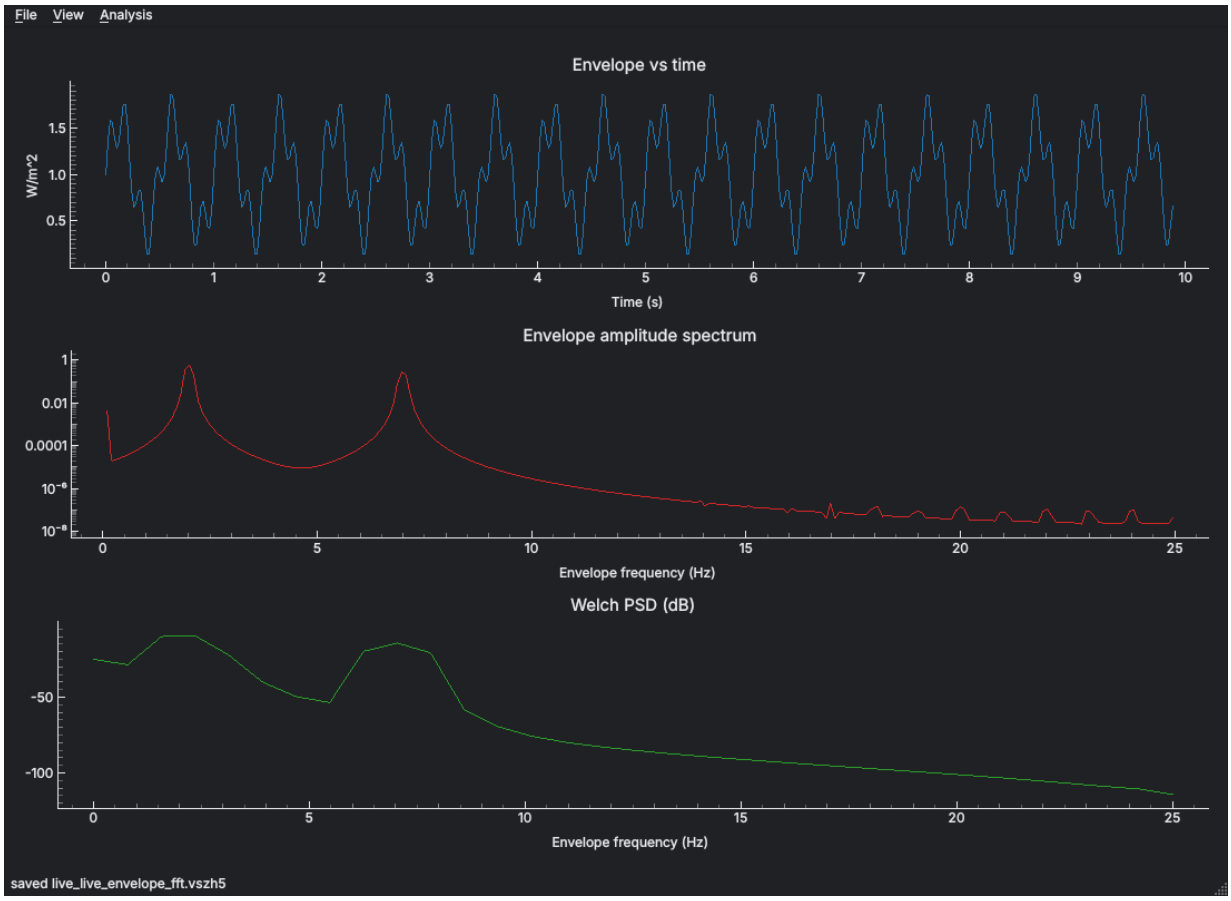


Figure 7: Live window (View → Dark) following a growing file with 2 Hz and 7 Hz envelope tones. *(Synthetic data.)*

```
series = load_any(r"C:\path\to\export.csv", opts)
result = analyse_timeseries(series, opts)
print(result.peaks[:5], result.families)
write_outputs(result, Path(r"C:\path\to\out"), opts)
flush_memory()
```

Table 10: Spyder cells of emf_envelope_fft_v0_0_2.py.

No.	Cell	Line	Definitions
1	MODULE HEADER	6	*(constants, imports or comments)*
2	IMPORTS	232	*(constants, imports or comments)*
3	CONSTANTS, PROBE CATALOGUE AND UNITS	295	ProbeSpec, lookup_probe, parse_unit, to_power_like, to_field_like
4	DATA MODEL	486	TimeSeries, SpectrumSet, Options
5	MEMORY HELPERS (bounded RAM for long logs)	582	_remove_spill_files, SpillBuffer, flush_memory, save_pickle, load_pickle
6	COMPUTE BACKEND (CPU / optional GPU)	695	Backend
7	TEXT EXPORT SNIFFING AND READING	736	TextLayout, _read_head, _is_number, _split, sniff_text, _pick_columns, TimeOrigin, _to_seconds, _origin_text, … (23 in all)
8	PRE-PROCESSING: TIMING, GAPS, UNIFORM GRID	1409	UniformSeries, make_uniform, analysis_quantity
9	SPECTRAL ESTIMATORS	1491	amplitude_spectrum, welch_blocks, czt_zoom, lomb_scargle, envelope_spectrogram, probe_response, correct_probe
10	PEAKS AND HARMONIC FAMILIES	1635	find_spectral_peaks, harmonic_families
11	ANALYSIS DRIVERS	1702	SeriesResult, _default_nperseg, analyse_timeseries, SpectrogramResult, _decimate_max, analyse_spectrumset
12	VEUSZ .vszh5 OUTPUT (through the author's vszh5_writer.py)	2028	_freq_unit, bookmark_outline, _write_series_pages, write_vszh5_series, write_vszh5_spectrogram
13	OTHER OUTPUTS: CSV, NPZ, JSON SUMMARY, OPTIONAL RENDER	2249	_summary_series, _summary_spectrogram, _jsonable, _write_csv, write_outputs, render_vszh5
14	BATCH (PARALLEL) PROCESSING	2364	process_file, process_files
15	LIVE MODE: FOLLOW A GROWING FILE	2411	LiveTail, RingBuffer, live_update, _live_ring, run_live_console, run_live_gui
16	SELF-TEST WITH SYNTHETIC EXPORTS	2743	_synthetic_radar, _probe_filter, make_synthetic_files, self_test
17	COMMAND LINE	2922	build_parser, options_from_args, main
18	INTERACTIVE SCRATCH CELL (Spyder)	3022	*(constants, imports or comments)*

11 Verification

`--selftest` checks the estimators against known answers and then runs every reader on synthetic files written in the styles expected from the instruments (semicolon and decimal comma with German dates, a 1 s NBM timer log, an EHP-50 TS folder, a SignalShark-like HDF5 file and a converter CSV). Table 11 is the result of the run made while building this manual.

Table 11: Self-test result (program 0.0.2, run during this build).

Check	Measured	Result
streamed Welch == scipy.signal.welch	max rel err 3.6e-15	PASS
FFT amplitude of 0.8 sinusoid	0.802	PASS
CZT peak frequency	0.21000 Hz	PASS
Lomb-Scargle amplitude and frequency	0.799 at 0.2100 Hz	PASS
|sinc| at 1.5 Hz, T=265 ms	-2.39 dB	PASS

Table 11 (continued)

Check	Measured	Result
FieldMan/TSX	radar 12 rpm family: [12.0, 42.0]	PASS
FieldMan/TSX	0.7 Hz source found	PASS
NBM timer log	6 rpm: [6.0]	PASS
EHP-50 TS spectra folder	0.05 Hz modulation of 50 Hz line: [0.05, 0.05]	PASS
signalshark	per-channel envelope 0.5 and 3 Hz: [0.5, 3.0]	PASS
signalshark_csv	per-channel envelope 0.5 and 3 Hz: [0.5, 3.0]	PASS
.vszh5 layout (Veusz format 1)	signalshark_converted_envelope_fft.vszh5	PASS

Further checks made for version 0.0.2:

- Every summary and CSV of the self-test compared with version 0.0.1: all numbers identical (two summary keys of the spectrogram result were renamed, see Appendix B).
- Rendering in Veusz 4.2.1 of a probe log (6 pages) and a SignalShark spectrogram (16 pages): date axis, line styles, markers, colour bars and the bookmark tree checked by eye and with pypdf.
- Live mode, console and window (light and dark), following a file written at 50 samples per second with 2 Hz and 7 Hz tones: both found; snapshot saved.
- A 2.4-million-row 900 Hz log through the spill-to-disk path: lines at 37.0 and 120.0 Hz found exactly.
- Three files with `--jobs 3 --gpu` on a machine without CuPy: all analysed, CPU fallback reported.

12 Limitations and known issues

1. The readers are heuristic until real exports of FieldMan, Narda-TSX, NBM-TS, EHP-50 TS and SignalShark have been examined; use `--inspect`.
2. “EFD-4091E” is treated as the EFD-4091; no datasheet for an E variant was found.
3. The probe averaging is modelled as a boxcar of the nominal integration time; the EF 0691’s integration time is not published, so $1/f_s$ is assumed.
4. The probe correction switches from the exact inverse to the Wiener-type gain at $|H| = 1/g_{\max}$ with a step of 1.94 dB (Section 5.11); it matters only for probes whose first null lies below their Nyquist frequency. A smooth transition is planned.
5. Content above the Nyquist frequency aliases; nothing in the data can remove it after the fact.
6. Above the True-RMS range the diodes leave square-law operation and powers no longer add; the envelope spectrum is then distorted (a warning is given).
7. The time page plots every sample, so very long logs make large PDFs.
8. Live mode depends on how the instrument software writes its files (Section 8).

13 Troubleshooting

Table 12: Troubleshooting.

Symptom	Likely cause	Remedy
’no value column found’	unusual column names	--value-col NAME (or index)
time axis wrong or ’cannot parse’	unusual date format, month/day order	--time-col, --dayfirst; check with --inspect

Table 12 (continued)

Symptom	Likely cause	Remedy
numbers 1000x too large	decimal comma read as thousands	--decimal , --sep ;
'measured sample rate differs'	logging interval longer than the probe rate	expected with timer logs; Nyquist is then lower
lines at 2f, f1+f2	--quantity field	use the default --quantity power
SignalShark axes look wrong	unknown .h5 layout	--inspect, then --h5-level/--h5-freq/--h5-time
dBm levels, no W/m^2	no antenna factor	--antenna-factor-db AF
--render: 'Veusz not found'	Veusz not on PATH	set VEUSZ_EXE to veusz.exe; the .vszh5 is written anyway
ImportError: vszh5_writer too old	an older copy imported first (Spyder)	restart the kernel; keep 0.0.11 next to the script
--gpu but 'CPU' in backend	no CuPy or CUDA device	mamba install -c conda-forge cupy; NVIDIA only
live: 'waiting for data'	file has no complete row yet, or other layout	check the file grows; --inspect it once

A Python modules used

This appendix lists every Python module the analyzer uses, the functions, classes and methods it calls, how it calls them and why that module was chosen. Table 13 is the overview; the following subsections give the details, module by module. Versions are those of the tested environment (2026-10-15).

Table 13: Python modules used by the analyzer.

Module	Tested	Needed for	Why this module
`numpy`	2.5.4	all arrays and array maths	common array type of every library used
`scipy`	1.18.1	FFT, windows, CZT, short-time FFT, peaks, interpolation	tested implementations of every estimator needed; scikit-rf has none of them
`pandas`	3.0.6	reading text and Excel exports	fast chunked CSV reader, tolerant date parsing
`openpyxl`	3.1.5	.xlsx (engine of pandas)	pandas' reader for .xlsx
`h5py`	3.16.0	SignalShark .h5; .vszh5 output	direct HDF5 access incl. references
`vszh5_writer`	0.0.11	Veusz documents, rendering	the author's Veusz writer
`Qt.py + PySide6`	2.0.7 / 6.12	live window	one Qt API, PySide6 binding
`pyqtgraph`	0.14.0	live plots	fast repeated plot updates
`cupy`	optional	--gpu	NumPy interface on NVIDIA GPUs
`pypdf`	6.20.1	PDF bookmarks (--render, manual check)	pure Python PDF editing
`veusz`	4.2.1	rendering (--render)	the plotting program of the documents
standard library	3.12.13	CLI, files, processes, time, JSON, ...	no extra dependency

A.1 NumPy

NumPy [20] holds every array of the program. It was chosen because SciPy, pandas, h5py, CuPy and pyqtgraph all exchange data as NumPy arrays, so no conversion is needed anywhere.

Table 14: NumPy calls.

Call	Where (function)	How and why
`np.asarray, np.ascontiguousarray`	everywhere	convert inputs to float64; contiguous arrays for FFTs and HDF5 writes
`np.argsort(kind='stable'), np.unique`	make_uniform, read_signalshark_csv	sort time stamps keeping equal stamps in order, drop duplicates; pivot long-format CSV
`np.diff, np.median, ndarray.mean`	make_uniform, analyse_spectrumset	sample interval = mean of gap-free intervals; jitter = median deviation
`np.interp`	make_uniform, read_ehp_folder	linear interpolation onto the uniform grid / common frequency axis: local, no overshoot, O(N)
`np.memmap, ndarray.tofile`	SpillBuffer, RingBuffer	long columns live in a raw file mapped into memory, so RAM stays bounded; live log on disk
`np.sinc`	probe_response	normalised sinc sin(pi x)/(pi x), the boxcar response of eq. (sinc)
`np.log10, np.maximum`	all dB conversions	dB values with a floor, so log10(0) never occurs
`np.nanmean, np.nanmax, np.nansum, np.where`	analyse_spectrumset	spectrogram statistics with missing cells (NaN) ignored
`np.stack, np.column_stack, np.vstack`	welch_blocks, readers, outputs	batch of Welch segments; tables for CSV; spectra of an EHP folder
`np.linspace, np.arange`	czt_zoom, lomb_scargle, synthetic files	frequency grids and time axes

Table 14 (continued)

Call	Where (function)	How and why
`np.convolve`	_probe_filter (self-test)	boxcar-average synthetic data like a probe
`np.savetxt, np.savez_compressed`	_write_csv, write_outputs	CSV and compressed .npz outputs
`np.random.default_rng`	self-test	reproducible synthetic noise and jitter

A.2 SciPy

SciPy [10] supplies the spectral estimators. It provides, in one tested library, everything the problem needs: real FFTs with multithreading, standard windows, the chirp-Z zoom, the short-time FFT and peak finding. scikit-rf [15] was considered and not used: it is built for network parameters (S-parameters, Touchstone files, calibration), not for spectral analysis of time series.

Table 15: SciPy calls.

Call	Where (function)	How and why
`scipy.fft.rfft(x, n, axis, workers)`	amplitude_spectrum, welch_blocks, analyse_spectrumset	FFT of real data (half the work of a complex FFT); workers = threads (-1 all, 1 per process in batch mode)
`scipy.fft.rfftfreq(n, d)`	same	frequency of every rfft bin
`scipy.signal.get_window(name, N)`	every estimator	any SciPy window by name (--window); periodic form (fftbins=True), correct for spectral analysis
`scipy.signal.zoom_fft(x, [f1, f2], m, fs, endpoint=True)`	czt_zoom	chirp-Z zoom (Bluestein), dense spectrum samples in a band
`scipy.signal.ShortTimeFFT(win, hop, fs, fft_mode='onesided2X', scale_to='psd')`	envelope_spectrogram	spectrogram(detr='constant', p0, p1), lower_border_end, upper_border_begin, t(), f: one-sided PSD slices without the slices hanging over the ends (SciPy >= 1.12)
`scipy.signal.find_peaks(x, height, prominence)`	find_spectral_peaks, analyse_spectrumset	local maxima above a level and prominence: envelope lines; carriers for channels
`scipy.signal.welch(...)`	self_test only	reference to which welch_blocks is compared
`scipy.interpolate.interp1d(x, y, axis=0)`	analyse_spectrumset	all frequency bins of a spectrogram onto a uniform time grid in one call (a legacy SciPy class, still supported)

A.3 pandas and openpyxl

pandas [21] reads the text and Excel exports. Its CSV reader is fast (C engine), reads in chunks, and its date parser copes with the many date and time formats instrument software writes [33]. openpyxl [34] is not imported by the program; pandas uses it as the engine for `.xlsx` files.

Table 16: pandas calls.

Call	Where (function)	How and why
`pd.read_csv(sep, decimal, encoding, skiprows, header=None, usecols, dtype=str, chunksize, engine, on_bad_lines='skip')`	read_text_timeseries, read_text_spectrum, read_signalshark_csv	reads only the needed columns as text, in chunks (bounded memory); bad lines skipped; numbers converted afterwards so decimal commas and thousands dots are handled
`Series.str.strip/.replace/.match`	_num, _to_seconds	clean quotes and spaces; decimal comma
`pd.to_numeric(errors='coerce')`	_num, _to_seconds	text to numbers, NaN where not a number
`pd.to_datetime(format='mixed', dayfirst, errors='coerce')`	_to_seconds, LiveTail.poll	date/time strings of any common format; .dt.tz_convert(None), .dt.total_seconds() give seconds
`pd.read_excel(sheet_name=0, header=None, dtype=str)`	read_xlsx_timeseries	first sheet as text (openpyxl engine), then the CSV reader path

Table 16 (continued)

Call	Where (function)	How and why
`DataFrame.to_csv(sep=';')`	read_xlsx_timeseries	temporary CSV of the sheet

A.4 h5py

h5py [22] reads SignalShark `.h5` files and, through `vszh5_writer.py`, writes the Veusz `.vszh5` documents, which are HDF5 files. It gives direct access to groups, datasets, attributes and object references, which the Veusz format needs.

Table 17: h5py calls.

Call	Where (function)	How and why
`h5py.File(path, 'r')`	read_signalshark_h5, describe_h5, self_test	open HDF5 read-only
`File.visititems(callback)`	read_signalshark_h5, describe_h5	walk every group and dataset to find level and axis datasets / print the tree
`Dataset.ndim/.shape/.dtype, .attrs, .parent`	read_signalshark_h5	choose datasets by shape and type; gather attributes up to the root (units, start/stop)
`h5py.File(path, 'w'), create_group, create_dataset(compression='gzip', compression_opts=4, shuffle=True)`	vszh5_writer. VeuszWriter	the .vszh5 layout, gzip for arrays of 4096+ values
`Dataset.ref (object reference)`	vszh5_writer. VeuszWriter.dataset_2d	links an image to its axis arrays (vsz_xrange etc.), as Veusz itself does

A.5 vszh5_writer (the author's module, 0.0.11)

`vszh5_writer.py` writes Veusz's native HDF5 document format without Veusz, and renders it to PDF and PNG when Veusz is installed. It came from the author's `isp_speed_monitor.py` and the QuickSyn package (0.0.10); for this program it was extended to 0.0.11 with two-dimensional datasets and plot-page helpers, without changing any existing call; the 2-D layout follows the Veusz source [29]. The analyzer uses it for every Veusz document, so the pages look the same as in the author's other programs.

Table 18: vszh5_writer calls.

Call	Where (function)	How and why
`VeuszWriter(path, compress=True, writer_tag)`	write_vszh5_series, write_vszh5_spectrogram	context manager; writes <path>.part, publishes with finish(), deletes on error (abort)
`dataset_1d / dataset_date / dataset_text`	_write_series_pages	typed Veusz datasets: numbers, dates (seconds since 2009-01-01), the JSON summary
`dataset_2d(name, data, xcent=, ycent=)`	_write_series_pages, write_vszh5_spectrogram	images; regular centres stored as xrange/yrange like Veusz (new in 0.0.11)
`VszDoc().text()`	write_vszh5_*	the Veusz command script stored in the document
`Trace(x, y, label, style, marker, line)`	_write_series_pages	one line or marker series of a page (new in 0.0.11)
`xy_page(doc, page, traces, title, xlabel, ylabel, xlog, ylog, xmin, xmode, x_autorange)`	_write_series_pages	a complete line-plot page with title and legend (new in 0.0.11)
`image_page(doc, page, dataset, title, ..., zmin, zmax)`	_write_series_pages, write_vszh5_spectrogram	a complete image page with colour bar (new in 0.0.11)

Table 18 (continued)

Call	Where (function)	How and why
`render_pages(path, png_pages='all', bookmarks=callable, pdf_info)`	render_vszh5	PDF and PNGs through Veusz; bookmarks from bookmark_outline()

A.6 Qt.py, PySide6 and pyqtgraph

The live window is written against Qt.py [24], a thin layer that exposes one Qt API over PySide6 or PyQt; the program asks for PySide6 [25] (`QT_PREFERRED_BINDING`). pyqtgraph [26] draws the plots: it is built for fast, repeated updates of large arrays, which a live display needs, and runs on the same Qt binding.

Table 19: Qt.py, PySide6 and pyqtgraph calls.

Call	Where (function)	How and why
`from Qt import QtCore, QtGui, QtWidgets`	run_live_gui	Qt.py picks the binding named in QT_PREFERRED_BINDING (PySide6)
`QApplication(), .setStyle('Fusion'), .setPalette(), .exec()`	run_live_gui, apply_theme	event loop; Fusion style so a palette themes every widget; Light/Dark
`QMainWindow.menuBar(). addMenu, .statusBar(), setCentralWidget`	LiveWindow	File/View/Analysis menus and the status line
`QActionGroup (QtGui in Qt 6, QtWidgets in Qt 5)`	LiveWindow._build_menus	exclusive Light/Dark choice; looked up in either module
`QtCore.QTimer.timeout, QTimer.singleShot`	LiveWindow	periodic update; --live-seconds
`QtGui.QPalette, QColor; style().standardPalette()`	apply_theme	dark and light palettes
`pyqtgraph. GraphicsLayoutWidget.addPlot`	LiveWindow	three stacked plots
`PlotItem.plot, PlotDataItem. setData, setLogMode, setLabel, mkPen`	LiveWindow.tick	fast redraw of the curves every update; log amplitude axis
`setBackground, getAxis(). setPen/.setTextPen`	apply_theme	plot colours of the theme

A.7 CuPy (optional)

CuPy [27] offers the NumPy interface on NVIDIA GPUs, so the same code runs on either device through the program's `Backend` class (its `xp` attribute is `numpy` or `cupy`). It is imported only when `--gpu` is given.

Table 20: CuPy calls (only with –gpu).

Call	Where (function)	How and why
`cupy.cuda.runtime. getDeviceCount()`	Backend	is a CUDA device present?
`cupy.asarray, cupy.fft.rfft`	Backend.asarray, Backend.rfft	arrays and FFTs on the GPU
`ndarray.get()`	Backend.cpu	copy results back to host memory
`get_default_memory_pool(). free_all_blocks(), pinned pool`	flush_memory	release GPU memory

A.8 pypdf and Veusz (rendering only)

Neither is imported by the analyzer itself; `vszh5_writer.py` uses them when `--render` is given, and the manual builder uses pypdf [28] to check the bookmarks of this PDF.

Table 21: pypdf and Veusz (through vszh5_writer).

Call	Where (function)	How and why
`veusz --listen --quiet ( script on stdin)`	vszh5_writer.render_ pages	loader.loadHDF5Doc then Export(): veusz --export cannot open .vszh5 files
`pypdf.PdfWriter(clone_from), add_outline_item, add_ metadata, page_mode`	add_pdf_bookmarks	bookmark tree and document information written into the rendered PDF
`pypdf.PdfReader(...).outline`	build_manual_pdf. verify_bookmarks	reads this manual's bookmarks back to check them

A.9 Python standard library

Table 22: Python standard library modules used.

Module	Calls	Purpose
`argparse`	ArgumentParser, add_argument	command line (also read by the manual builder)
`atexit`	register	delete spill files when the program exits
`concurrent.futures`	ProcessPoolExecutor, as_completed	--jobs: files in parallel processes
`csv`	reader	split lines with quoted fields
`dataclasses`	dataclass, field, asdict	Options, TimeSeries, results
`datetime`	datetime, timedelta, timezone	time origins, EHP file times, Veusz dates
`gc`	collect	flush_memory()
`json`	dumps	summary file and summary dataset
`math`	floor, ceil, log2, sqrt, isfinite	segment lengths, scalars
`os`	fdopen, unlink, replace, environ	spill files, atomic writes, Qt binding choice
`pathlib`	Path	all file paths (Windows and Linux)
`pickle`	dump (protocol 5), load	--pickle offload of results
`re`	search, match, fullmatch, compile	units, headers, file names, bookmarks
`sys`	path, modules, argv, exit	module path for vszh5_writer, CuPy check, CLI
`tempfile`	mkstemp, mkdtemp	spill files, xlsx conversion, self-test folder
`time`	perf_counter, time, sleep	timing reports, live polling
`typing`	Any, Sequence	type hints
`warnings`	warn(RuntimeWarning)	render problems reported without stopping
`subprocess, threading`	Popen, Timer	vszh5_writer: drive Veusz with a timeout

B Revision history

Table 23: Revision history of the program, the writer module and this manual.

Version	Date	Changes
`0.0.1`	2026-10-15	First draft of the analyzer: readers, FFT, Welch, CZT, Lomb-Scargle, spectrogram, probe model and correction, harmonic families, per-bin modulation, .vszh5 output, batch, GPU option, live mode, self-test.

Table 23 (continued)

Version	Date	Changes
`0.0.2`	2026-10-15	Readability pass (descriptive names, detailed comments; numerical results identical). Veusz output through vszh5_writer 0.0.11; date axis on time pages; zoom page with FFT bins; hierarchical PDF bookmarks; summary keys rf_bins/rf_range_hz renamed frequency_bins/frequency_range_hz; fuller dark palette. This manual and its build tool.
`writer 0.0.11`	2026-10-15	vszh5_writer: dataset_2d, Trace, add_title, xy_page, image_page (manual colour bar, exact ranges), date axes; existing calls unchanged.

References

[1] Narda Safety Test Solutions GmbH, “Datasheet Probe EFD-4091,” NSTS 0523-E0399A, 2023. [Online]. Available: https://www.atecorp.com/getmedia/48a83a03-af45-41d0-820c-ecdb46ba1437/Narda_EFD-4091-datasheet.pdf

[2] Narda Safety Test Solutions GmbH, “Datasheet Probe EFD-0691,” NSTS 0624-E0387B, 2024. [Online]. Available: https://www.atecorp.com/getmedia/84e296e2-1bd2-4861-8a1f-e905f39d4910/Datasheet_EFD-0691.pdf

[3] Narda Safety Test Solutions GmbH, “Datasheet Probe EFD-9091,” NSTS 0523-E0405A, 2023. [Online]. Available: https://www.atecorp.com/getmedia/c56e0776-d855-4d49-9081-ff316ecaf68a/Datasheet_EFD-9091.pdf

[4] Narda Safety Test Solutions GmbH, “Datasheet Probe EFD-1891,” NSTS 0723-E0398A, 2023. [Online]. Available: https://www.atecorp.com/getmedia/d5ed148e-2dfc-4f3c-afa4-2b306fb7a781/Narda_EFD-1891-datasheet.pdf

[5] Narda Safety Test Solutions GmbH, “Datasheet Narda FieldMan,” NSTS 0925-E0370D, 2025. [Online]. Available: https://www.atecorp.com/getmedia/5511a8ac-b386-434f-bac9-fd22d18e2f48/Narda-Fieldman-Datasheet-2025.pdf

[6] Narda Safety Test Solutions, “NBM-550 Remote Control,” V1.0. [Online]. Available: https://wiki.radimation.com/wiki/images/7/7b/NBM-550_Remote_Control_V1_0.pdf

[7] Narda Safety Test Solutions S.r.l., “EHP-50F User’s Manual,” EHP50FEN-10107-1.73, 2021. [Online]. Available: https://atecorp.com/getmedia/d53b58ae-844a-4a2d-b84e-6a9f98be805a/narda-ehp-50f-operating-manual.pdf

[8] Narda Safety Test Solutions, “SignalShark options and scripts.” [Online]. Available: https://www.narda-sts.com/en/products/receivers-/-direction-finders-/-spectrum-analyzers/signalshark-optionen/

[9] Narda Safety Test Solutions, “PC software Narda-TSX.” [Online]. Available: https://www.narda-sts.com/en/products/emc-products-and-streaming-solutions/pc-software-narda-tsx/

[10] P. Virtanen *et al.*, “SciPy 1.0: Fundamental algorithms for scientific computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.

[11] P. D. Welch, “The use of fast Fourier transform for the estimation of power spectra: A method based on time averaging over short, modified periodograms,” *IEEE Trans. Audio Electroacoust.*, vol. AU-15, no. 2, pp. 70–73, Jun. 1967, doi: 10.1109/TAU.1967.1161901.

[12] L. R. Rabiner, R. W. Schafer, and C. M. Rader, “The chirp z-transform algorithm,” *IEEE Trans. Audio Electroacoust.*, vol. AU-17, no. 2, pp. 86–92, Jun. 1969, doi: 10.1109/TAU.1969.1162034.

[13] J. D. Scargle, “Studies in astronomical time series analysis. II. Statistical aspects of spectral analysis of unevenly spaced data,” *Astrophys. J.*, vol. 263, pp. 835–853, 1982, doi: 10.1086/160554.

[14] J. T. VanderPlas, “Understanding the Lomb–Scargle periodogram,” *Astrophys. J. Suppl. Ser.*, vol. 236, no. 1, 2018, doi: 10.3847/1538-4365/aab766.

[15] scikit-rf developers, “scikit-rf documentation.” [Online]. Available: https://scikit-rf.readthedocs.io/

[16] F. J. Harris, “On the use of windows for harmonic analysis with the discrete Fourier transform,” *Proc. IEEE*, vol. 66, no. 1, pp. 51–83, Jan. 1978, doi: 10.1109/PROC.1978.10837.

[17] *IEEE Standard for Calibration of Electromagnetic Field Sensors and Probes (Excluding Antennas) from 9 kHz to 40 GHz*, IEEE Std 1309-2013. [Online]. Available: https://standards.ieee.org/ieee/1309/5031

[18] *Determination of RF field strength, power density and SAR in the vicinity of base stations for the purpose of evaluating human exposure*, IEC 62232:2025. [Online]. Available: https://webstore.iec.ch/en/publication/89073

[19] L. I. Bluestein, “A linear filtering approach to the computation of discrete Fourier transform,” *IEEE Trans. Audio Electroacoust.*, vol. 18, no. 4, pp. 451–455, Dec. 1970, doi: 10.1109/TAU.1970.1162132.

[20] C. R. Harris *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[21] W. McKinney, “Data structures for statistical computing in Python,” in *Proc. 9th Python in Science Conf. (SciPy 2010)*, S. van der Walt and J. Millman, Eds., 2010, pp. 56–61, doi: 10.25080/Majora-92bf1922-00a.

[22] h5py developers, “HDF5 for Python (h5py) documentation.” [Online]. Available: https://docs.h5py.org/

[23] J. Sanders *et al.*, “Veusz — a scientific plotting package.” [Online]. Available: https://veusz.github.io/

[24] M. Ottosson *et al.*, “Qt.py: Minimal Python 2 & 3 shim around all Qt bindings.” [Online]. Available: https://github.com/mottosso/Qt.py

[25] The Qt Company, “Qt for Python (PySide6) documentation.” [Online]. Available: https://doc.qt.io/qtforpython-6/

[26] pyqtgraph developers, “PyQtGraph: Scientific graphics and GUI library for Python.” [Online]. Available: https://www.pyqtgraph.org/

[27] R. Okuta, Y. Unno, D. Nishino, S. Hido, and C. Loomis, “CuPy: A NumPy-compatible library for NVIDIA GPU calculations,” in *Proc. Workshop on Machine Learning Systems (LearningSys), 31st Conf. Neural Information Processing Systems (NIPS)*, 2017. [Online]. Available: http://learningsys.org/nips17/assets/papers/paper_16.pdf

[28] pypdf developers, “pypdf documentation.” [Online]. Available: https://pypdf.readthedocs.io/

[29] Veusz developers, “Veusz 4.2.1 source (document/doc.py, document/loader.py, datasets/twod.py),” 2024. [Online]. Available: https://pypi.org/project/veusz/4.2.1/

[30] SciPy developers, “scipy.signal.ShortTimeFFT.” [Online]. Available: https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.ShortTimeFFT.html

[31] SciPy developers, “scipy.signal.zoom_fft.” [Online]. Available: https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.zoom_fft.html

[32] P. J. Mohr, D. B. Newell, B. N. Taylor, and E. Tiesinga, “CODATA recommended values of the fundamental physical constants: 2022,” arXiv:2409.03787, 2024. [Online]. Available: https://arxiv.org/abs/2409.03787

[33] pandas developers, “pandas documentation.” [Online]. Available: https://pandas.pydata.org/docs/

[34] openpyxl developers, “openpyxl documentation.” [Online]. Available: https://openpyxl.readthedocs.io/

[35] Spyder project, “Spyder — the scientific Python development environment.” [Online]. Available: https://www.spyder-ide.org/

[36] conda-forge, “Miniforge (conda and mamba).” [Online]. Available: https://github.com/conda-forge/miniforge

[37] H. Oberdiek and the LaTeX Project, “The bookmark package.” [Online]. Available: https://ctan.org/pkg/bookmark